

Interview Explanation

Milestone 6 - D435i SLAM/Nav2 Plan for MuJoCo G1 Maze Navigation

Purpose: explain what I wanted to build, how I directed the work, what problems came up, and how I planned the next technical milestones.

One-sentence interview framing

I started from a MuJoCo Unitree G1 maze-navigation simulation, stabilized the development environment with Docker + ROS 2 Humble, then designed a staged D435i-style sensing pipeline: simulated RGB-D + IMU topics, depth-to-scan conversion, SLAM, Nav2, and KPI reporting, while keeping oracle data only for labeled debugging and evaluation.

Area	Decision
Simulation engine	MuJoCo remains the physics simulator and main visual demo environment.
ROS role	ROS 2 Humble, RViz, SLAM toolbox, depthimage_to_laserscan, and Nav2 are used for perception/navigation introspection and autonomous planning.
Sensor direction	Simulate a D435i-style RGB-D + IMU sensor in MuJoCo and publish RealSense-like topics.
Navigation direction	Use Nav2 planning from a SLAM/costmap representation, not an oracle maze path.
Evaluation direction	Keep oracle/ground truth only for scoring, top-down truth visualization, and debug comparisons.

1. Project objective

The assignment goal is to make a Unitree G1 humanoid navigate a randomly generated maze in MuJoCo, collect data, and produce meaningful KPIs for a live demo. The project had already reached a working locomotion-oriented milestone using a Lucky Robots G1 walking policy and visual artifacts, but it still needed the more autonomous navigation layer: sensing, SLAM, Nav2, and reporting.

- Use seeded mazes so the same seed reproduces the same environment, while still supporting random live-demo seeds.
- Use a real-world-plausible sensor setup: a D435i-style depth camera and IMU mounted on the simulated robot.
- Use ROS 2 navigation tools instead of hand-coded oracle path following for the autonomous milestone.
- Maintain visible artifacts after every step: rendered images, dashboards, top-down trajectories, RViz screenshots, logs, and summary JSON files.
- Keep the implementation defensible: if the robot uses oracle information, it must be labeled as debug/evaluation only, not hidden inside navigation.

2. How I directed the work

I treated AI/Codex as an implementation partner, not as a black-box replacement for engineering judgment. The main direction was to keep the project staged, inspectable, and testable. Instead of asking for a large, vague implementation, I turned the work into small milestones with strict visible outputs and validation commands.

Direction style	Reason
Plan first, execute one phase at a time	Prevents Codex from changing too much and makes failures easier to isolate.
Visible artifacts required	The project is simulation-heavy; screenshots, dashboards, maps, and trajectories are necessary to prove behavior.
Do not rewrite existing logic	Docker/ROS infrastructure should not break existing MuJoCo milestones.
Ground truth separated from navigation	Keeps the autonomous claim honest while still allowing strong KPI analysis.
Docker as development environment	Allows continued SLAM/Nav2/KPI work without depending on native ROS installation.

Interview point

A strong part of the work was not only coding, but controlling scope: I separated model/rendering work, ROS bridge work, scan conversion, SLAM, Nav2, and KPI reporting so every layer could be validated before building the next one.

3. Environment decision: Docker + Ubuntu 22.04 + ROS 2 Humble

The project needed to run on different machines, including regular laptops and potentially Jetson/ARM machines. I did not want to depend on ROS being installed on the host, especially because the host already had Ubuntu 20.04 and ROS Noetic context from earlier work. The chosen approach was to put ROS 2 Humble inside Docker.

Option	Decision
Ubuntu 22.04 + ROS 2 Humble	Chosen for stability and compatibility with Nav2 and ROS 2 Humble packages.
Ubuntu 24.04 + ROS 2 Jazzy	Rejected for now because newer does not help the assignment enough to justify migration/debug risk.
Host ROS installation	Not required. Docker contains the full ROS 2 environment.
GPU support	Postponed. Jetson GPU and x86 NVIDIA GPU paths differ too much for the base portable image.
Multi-architecture support	Buildx support was requested for linux/amd64 and linux/arm64.

Host machine:

Docker installed
ROS 2 not required

Docker container:

Ubuntu 22.04
ROS 2 Humble
Nav2
slam_toolbox
depthimage_to_laserscan
robot_localization
rviz2
MuJoCo/project dependencies

4. Docker validation and problems encountered

After creating the Docker setup, I validated the important ROS components inside the container. The checks confirmed that ROS 2 Humble, Nav2, slam_toolbox, depthimage_to_laserscan, and RViz were available.

- ROS_DISTRO printed humble.
- Nav2 packages appeared in ros2 pkg list, including planner/controller/costmap/rviz plugins.
- slam_toolbox and depthimage_to_laserscan were installed.
- RViz launched successfully in the GUI container.
- OpenGL 4.6 was reported, meaning GUI/OpenGL passthrough worked.

Problem	Interpretation	Resolution / next action
RViz warning: XDG_RUNTIME_DIR not set	Minor environment warning, not fatal.	Can clean up in run_gui.sh later, but not blocking.
RViz message: Stereo is NOT SUPPORTED	Normal RViz/OpenGL message; not related to D435i stereo data.	Ignore unless using stereo display mode.
docker command not found inside container	make docker-check-ros was called from inside Docker and tried to	Fix Makefile to detect /.dockerenv and run the check directly when

Milestone 6 Interview Explanation - D435i SLAM/Nav2 Plan

	launch Docker again.	already inside the container.
GPU portability question	Jetson and laptop GPU setups are different.	Postpone GPU and focus on portable CPU/GUI Docker first.

5. D435i-style sensing architecture

The D435i is the right conceptual sensor for this milestone because it provides depth/RGB and IMU-style data. However, in MuJoCo simulation there is no physical USB RealSense device, so the simulation should not run `realsense2_camera`. Instead, MuJoCo should simulate the camera and publish RealSense-like ROS 2 topics.

```
MuJoCo G1 + simulated D435i
-> ROS 2 sensor bridge
-> /camera/depth/image_rect_raw
-> /camera/color/image_raw
-> /camera/depth/camera_info
-> /camera/color/camera_info
-> /imu/data
-> /tf, /tf_static, /joint_states, /clock
-> depthimage_to_laserscan
-> /scan
-> slam_toolbox
-> /map
-> Nav2
-> /cmd_vel
-> G1 walking-policy adapter
-> MuJoCo controls
```

Key design point

In simulation, the project publishes D435i-like topics. On real hardware, that simulated publisher could later be replaced by the real RealSense ROS driver, but the downstream ROS stack should remain similar.

6. Why RViz is useful even though the robot runs in MuJoCo

MuJoCo and RViz serve different purposes. MuJoCo shows the physical simulation. RViz shows what the robot navigation stack believes.

Tool	What it answers
MuJoCo	What is physically happening? How is the G1 walking? Did it collide? What does the simulated world look like?
RViz	Are TF frames correct? Is <code>/scan</code> aligned? Is the SLAM map sane? What path did Nav2 plan? What does the costmap believe?

- If the robot hits a wall in MuJoCo, RViz can reveal whether `/scan` is rotated, the costmap is shifted, the footprint is wrong, or odometry is drifting.
- RViz is not the final simulator; it is the diagnostic view into ROS perception and navigation.
- The best demo/report will show both: MuJoCo truth and RViz robot belief.

7. Milestone 6 - seven-phase plan

Phase	Purpose	Main validation artifact
0. Docker stable	Finish the portable Ubuntu 22.04 + ROS 2 Humble development environment.	docker-check-ros, RViz launch, test/smoke results.
1. Simulated D435i mount	Add a D435i-style RGB-D + IMU sensor to the G1/MuJoCo model.	RGB/depth camera images and camera mount dashboard.
2. ROS 2 sensor bridge	Publish camera, depth, CameraInfo, IMU, TF, joint_states, and clock from MuJoCo.	ROS topic list, rates, TF tree, RGB/depth snapshots.
3. Depth to /scan	Convert depth image into LaserScan using depthimage_to_laserscan.	/scan visible in RViz and aligned with maze walls.
4. SLAM	Use slam_toolbox to build an occupancy map from /scan.	Saved map, RViz map screenshot, map quality metrics.
5. Nav2 in SLAM mode	Let Nav2 plan from the SLAM/costmap data and produce /cmd_vel.	Nav2 path, costmap, command timeline, dashboard.
6. Navigate to goal	Connect the mission goal and drive the G1 to the maze exit.	Goal reached/failed status, trajectory, final render.
7. KPI reporting	Run multi-seed evaluation and produce reporting dashboards.	summary.csv/json, report.html, success/time/path/contact metrics.

8. Important phase boundaries

The phase boundaries were intentionally strict. For example, /scan is not part of Phase 2 because it is not a raw D435i output. Phase 2 publishes depth and CameraInfo. Phase 3 converts those topics into /scan.

Phase 2:

Publish raw simulated sensor topics:

```
/camera/depth/image_rect_raw
/camera/depth/camera_info
/camera/color/image_raw
/imu/data
/tf
/joint_states
/clock
```

Phase 3:

```
Run depthimage_to_laserscan:
depth + CameraInfo -> /scan
```

- This makes debugging easier because an empty /scan could come from depth rendering, CameraInfo, TF, remapping, or the converter itself.
- By validating each layer separately, I can isolate failures instead of debugging a large combined system.
- This is especially important in ROS, where TF and frame conventions often cause subtle failures.

9. Oracle policy: what is allowed and what is not

A major design decision was to stop using oracle planning once the SLAM/Nav2 phase begins. Oracle mode is valuable as a baseline and evaluation tool, but it should not secretly control the final autonomous navigation mode.

Milestone 6 Interview Explanation - D435i SLAM/Nav2 Plan

Allowed oracle use	Not allowed in SLAM/Nav2 navigation
Ground-truth trajectory for KPI comparison.	Using the generated maze truth to choose the path.
Compare SLAM map against true generated maze.	Using MuJoCo ground-truth pose as normal odometry for navigation.
Measure wall contacts and true clearance.	Feeding perfect wall geometry into Nav2 while claiming sensor-based navigation.
Top-down truth visualization.	Driving toward precomputed oracle waypoints while saying Nav2 is in control.

Interview wording

Oracle mode remains as a labeled baseline and evaluation tool. In SLAM/Nav2 mode, the robot plans from its built map and sensor-derived localization, while ground truth is logged only for scoring and visualization.

10. Top-down visualization plan

A top-down view is necessary because it clearly shows whether the robot actually navigated the maze. I planned two complementary top-down views: truth and belief.

View	Data source	Purpose
Truth top-down	Generated maze geometry + MuJoCo ground-truth trajectory.	Shows how the robot actually walked, including contacts, recoveries, and final pose.
Robot-belief top-down	RViz/ROS map, scan, odom, TF, Nav2 path, and costmaps.	Shows what the robot thought was happening and how Nav2 planned.

Recommended Phase 5 artifacts:

```
runs/visual/nav2_seed-123_dashboard.html
runs/visual/nav2_seed-123_topdown_truth.svg
runs/visual/nav2_seed-123_topdown_belief.png
runs/visual/nav2_seed-123_rviz.png
runs/visual/nav2_seed-123_final_render.png
runs/visual/nav2_seed-123_trajectory.csv
runs/visual/nav2_seed-123_nav2_path.csv
runs/visual/nav2_seed-123_cmd_vel.csv
runs/visual/nav2_seed-123_summary.json
```

11. KPI/reporting plan

The KPI layer should prove not only that one demo seed worked, but how robust the approach is across seeds. It should also separate navigation performance from sensing/mapping performance.

KPI category	Examples
Navigation	Success rate, time to goal, path length, actual path / planned path, distance-to-goal over time.
Safety	Wall contacts, first contact time, minimum clearance, recovery events, stuck/timeout count.
SLAM/perception	Map coverage, scan rate, camera FPS, dropped frames, map quality vs true maze for evaluation only.
Control/locomotion	Commanded vs achieved velocity, yaw tracking, controller state timeline, /cmd_vel rate.
System/demo	Per-seed summary, HTML dashboard, plots, videos, reproducible run metadata.

12. Problems that occurred and how I handled them

Problem / question	Engineering response
Should ROS 2 Humble be installed before Docker?	No. Docker should contain ROS 2 Humble; the host only needs Docker.
Can Docker use GPU?	Yes in principle, but Jetson and x86 GPU paths differ. Postponed GPU to keep base image portable.
Ubuntu 22 or 24?	Use Ubuntu 22.04 + Humble for stable assignment progress; avoid Jazzy migration risk now.
How to avoid burning Codex tokens?	Use PDF/reference plus small phase-by-phase prompts. Plan first, execute one phase only.
Why RViz if using MuJoCo?	MuJoCo is physics truth; RViz is robot belief and ROS introspection. Both are needed.
Should oracle plan in Phase 5?	No. Oracle is allowed only for labeled debugging/evaluation once SLAM/Nav2 mode begins.
Why no /scan in Phase 2?	/scan is derived from depth by depthimage_to_laserscan, so it belongs to Phase 3.

13. How I used Codex/AI correctly

The AI workflow was intentionally controlled. I did not ask for a vague complete solution. I asked for infrastructure, then staged perception/navigation components, with explicit restrictions and validation criteria.

- I specified the target OS/ROS stack, Docker files, Makefile targets, and validation commands instead of leaving infrastructure ambiguous.
- I required visible artifacts and dashboards so results could be inspected without relying only on terminal output.
- I separated phases by failure domain: MuJoCo rendering, ROS bridge, scan conversion, SLAM, Nav2, goal navigation, and KPIs.
- I asked Codex to plan first and execute one phase at a time, reducing risk and token use.

Milestone 6 Interview Explanation - D435i SLAM/Nav2 Plan

- I kept debug/oracle features clearly labeled so the final navigation claim remains honest.

14. Interview talking points

Question	Strong answer
What were you trying to build?	A reproducible MuJoCo/ROS 2 pipeline where a Unitree G1 navigates a random maze using simulated D435i sensing, SLAM, Nav2, and a walking-policy adapter.
Why Docker?	To isolate ROS 2 Humble/Nav2 dependencies from the host and make the project reproducible across machines.
Why D435i?	It is a plausible RGB-D + IMU sensor for the robot, and depth can be converted into LaserScan for 2D maze SLAM.
Why start with depth-to-scan instead of full RGB-D SLAM?	The maze is mostly 2D with vertical walls, so depth-to-scan + slam_toolbox is simpler, robust, and easier to debug.
What was the main risk?	TF/CameraInfo/depth-frame correctness and odometry quality for a humanoid, plus making Nav2 assumptions work with a walking policy.
How did you avoid cheating?	Ground truth/oracle data is used only for evaluation and visualization, not for final SLAM/Nav2 planning.
How did you validate progress?	Each phase produces concrete artifacts: RGB/depth images, ROS topic rates, TF trees, /scan in RViz, maps, paths, trajectories, and dashboards.

15. Immediate next actions

- Fix the minor docker-check-ros UX bug so checks work both from the host and from inside Docker.
- Execute Phase 1 only: add the simulated D435i mount and generate RGB/depth visual artifacts.
- Validate Phase 1 with make d435i-visual-check SEED=123 and inspect the camera view.
- Execute Phase 2 only: publish ROS 2 camera/depth/CameraInfo/IMU/TF/joint_states/clock topics.
- Execute Phase 3 only: run depthimage_to_laserscan and verify /scan alignment in RViz.
- Only then proceed to slam_toolbox mapping, Nav2 planning, goal navigation, and KPI reporting.

16. Sixty-second project pitch

A concise interview answer: I am building a reproducible humanoid maze-navigation simulation around MuJoCo and ROS 2. I first stabilized the environment with Docker using Ubuntu 22.04 and ROS 2 Humble, so the project does not depend on native ROS installation. Then I designed the autonomous navigation stack in layers: a simulated D435i-style RGB-D + IMU camera mounted on the G1, a ROS bridge that publishes RealSense-like topics, depthimage_to_laserscan to produce /scan, slam_toolbox to build the map, and Nav2 to plan and send /cmd_vel to the G1 walking-policy adapter. I kept MuJoCo as the physical truth view and RViz as the robot-belief/debug view. I also separated oracle data from navigation: ground truth is used only for scoring, top-down visualization, and KPI comparison, not for deciding the final path. The key engineering approach was to build one validated layer at a time, with visible artifacts after every milestone.

17. Quick checklist before interview/demo

- Can I explain why Ubuntu 22.04 + ROS 2 Humble was chosen?
- Can I show that Docker/RViz/Nav2/slam_toolbox are installed and working?
- Can I explain why realsense2_camera is not used inside pure simulation?
- Can I explain the difference between MuJoCo truth and RViz robot belief?
- Can I show the phase plan and justify why /scan is Phase 3?
- Can I state exactly where oracle is used and where it is forbidden?
- Can I show or generate top-down trajectory artifacts?
- Can I define the KPI metrics I plan to collect across seeds?